



Flask

Aula 08: Sistema de login

Login, sessions, flash messages e middleware

A biblioteca **"Werkzeug"** é uma biblioteca em Python que fornece várias ferramentas para lidar com tarefas comuns da web, como manipulação de solicitações HTTP, manipulação de URLs e autenticação. Dentro da Werkzeug, a sub-biblioteca **"security"** é específica para lidar com questões relacionadas à segurança da web.

A biblioteca **Werkzeug Security** oferece recursos para **criptografia de senha, geração de hashes, manipulação de tokens de segurança** e outras funções relacionadas à segurança.

Isso é útil em aplicativos da web para proteger informações confidenciais e garantir a integridade dos dados transmitidos pela internet. Por exemplo, você pode usá-la para garantir que as senhas dos usuários sejam armazenadas com segurança em seu banco de dados, protegendo contra ataques de força bruta e outros tipos de ataques de segurança.



Werkzeug

Abaixo um exemplo de código onde é feito o cadastro do usuário e gerado um hash seguro da senha para armazenamento, impossibilitando a reversão, com a função `generate_password_hash`.

```
@app.route('/caduser', methods=['GET', 'POST'])
def caduser():
    if request.method == 'POST':
        email = request.form['email']
        password = request.form['password']
        hashed_password = generate_password_hash(password, method='scrypt')
        new_user = Usuario(email=email, password=hashed_password)

        db.session.add(new_user)
        db.session.commit()

        return redirect(url_for('login'))

    return render_template('caduser.html')
```

Flash messages



As **flash messages** no **Flask** são mensagens temporárias que são armazenadas e exibidas uma única vez para o usuário após uma determinada ação, como enviar um formulário ou realizar uma operação no servidor. Essas mensagens são frequentemente usadas para **fornecer feedback instantâneo** sobre o resultado de uma ação.

No Flask, você pode usar a funcionalidade de **flash messages** através do objeto **flash** fornecido pelo framework. Por exemplo, após o processamento bem-sucedido de um formulário, você pode usar **flash("Formulário enviado com sucesso!")** para armazenar a mensagem **"Formulário enviado com sucesso!"**. Depois, você pode exibir essa mensagem na próxima página renderizada.

Geralmente, as **flash messages** são usadas para informar o usuário sobre erros de validação de formulários, confirmação de ações realizadas com sucesso, ou qualquer outra informação que precise ser comunicada de forma temporária. Elas são especialmente úteis em aplicativos da web para fornecer uma **experiência de usuário mais interativa e responsiva**.

Success! Your message has been sent successfully.



Error! A problem has been occurred while submitting your data.



Warning! There was a problem with your network connection.



Info! Please read the comments carefully.



Info! Please read the comments carefully.



No **Flask**, as **sessions** são uma forma de armazenar dados do lado do servidor para um usuário específico entre solicitações HTTP. Elas permitem que você mantenha o estado da aplicação e informações do usuário enquanto ele navega pelo seu aplicativo.

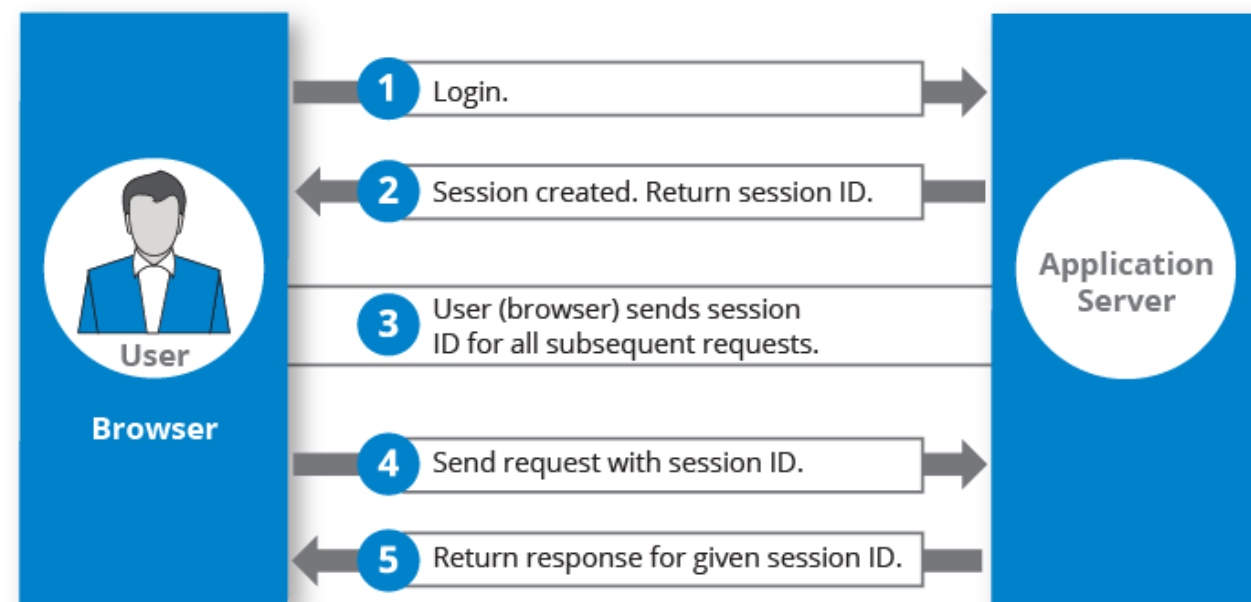
As **sessions** no **Flask** são implementadas usando **cookies**. Quando você inicia uma sessão, o Flask envia um cookie para o navegador do usuário que contém um identificador de sessão único. Esse identificador é usado para associar os dados da sessão armazenados no servidor ao usuário que está fazendo a solicitação.

SESSIONS IN FLASK



Você pode armazenar qualquer tipo de dados na sessão, como informações de login do usuário, preferências do usuário, carrinho de compras em uma loja online, etc. As **sessions** são uma maneira conveniente de persistir temporariamente dados do lado do servidor para um usuário específico durante sua interação com o aplicativo.

No **Flask**, você pode acessar os dados da sessão usando o objeto **session**. Por exemplo, para definir um valor na sessão, você pode usar **session['nome'] = 'valor'**, e para recuperar um valor, você pode usar **valor = session.get('nome')**.



Abaixo um exemplo de código, onde é feito a checagem do hash da senha do usuário com a função `check_password_hash` e criado a sessão para o usuário.

```
@app.route('/login', methods=['GET', 'POST'])
def login():
    if request.method == 'POST':
        email = request.form['email']
        password = request.form['password']

        user = Usuario.query.filter_by(email=email).first()

        if user and check_password_hash(user.password, password):
            session['user_id'] = user.id
            session['email'] = user.email
            nickname = user.email.split('@')
            flash(f'Login bem-sucedido! Bem-vindo {nickname[0]}!', 'success')
            return redirect(url_for('home'))
        else:
            flash('Falha no login. Verifique seu nome de usuário e senha.', 'danger')
    return render_template('login.html')
```

Middleware



Em **Flask**, um **middleware** é um componente intermediário que pode processar solicitações HTTP **antes que elas sejam tratadas pelas rotas** do aplicativo Flask. Isso permite que você injete lógica de processamento adicional em todas as solicitações ou respostas do aplicativo.

Um **middleware** em **Flask** pode ser usado, por exemplo, para **autenticar** usuários **antes de encaminhar a solicitação para as rotas** do aplicativo.

